

第5章 语法制导的翻译

Syntax-Directed Translation

编译原理·考点总结

本章重点：语法制导定义（SDD）、S属性/L属性定义、依赖图、SDT设计与实现、L属性自底向上实现

5.1 语法制导定义 (SDD)

基本思想 (P.4)

- 翻译任务：先语义分析与正确性检查，再生成中间代码或目标代码。
- 核心思想：给文法符号附加属性（如变量的类型、层次、存储地址；表达式的值、类型），将属性

语义分析的任务 (P.5)

- 语义检查：类型一致性、运算合法性、数组维数、下标越界等。
- 语义处理：变量存储分配、表达式求值、语句翻译（中间代码生成）。
- 总目标：生成等价的中间代码。

什么是SDD (P.10)

语法制导定义 (SDD, Syntax-Directed Definition) = 上下文无关文法 + 属性/语义规则：

- 属性与文法符号相关联；规则与产生式相关联。
- 语义规则描述如何计算属性值。例：

$$E \rightarrow E_1 + T \Rightarrow E.code = E_1.code \parallel T.code \parallel '+'$$

典型处理方法 (P.7--P.8)

1. 方法1（归约时执行）：每个产生式配一个语义子程序，产生式匹配时调用。适合自底向上分析。

$$E \rightarrow E_1 + T : E.val := E_1.val + T.val$$

2. 方法2（推导时执行）：在产生式体中插入语义动作，按分析进程执行。适合自顶向下分析。

$$D \rightarrow T \{L.in := T.type\} L$$

5.1.1 继承属性与综合属性 (P.12--P.14)

综合属性 (Synthesized Attribute) (P.12)

- 分析树结点 N 上非终结符 A 的属性值，由 N 上产生式关联的语义规则定义。
- 只能通过 N 的子结点或 N 本身的属性值来计算（自下而上流动）。
- 产生式 $A \rightarrow xB$ ，若 $A.a_1 = f(B.b)$ ，则 a_1 为综合属性。

继承属性 (Inherited Attribute) (P.13)

- 分析树结点 N 上非终结符 B 的属性，由 N 的父结点上的产生式关联规则定义。
- 依赖于 N 的父结点、 N 本身和 N 的兄弟结点的属性值（自上而下/从左到右流动）。
- 产生式 $A \rightarrow xB$ ，若 $B.b = f(A.a_1)$ ，则 b 为继承属性。

限制 (P.14)

- 不允许 N 的继承属性通过 N 的子结点来定义（防止循环）。
- 允许 N 的综合属性依赖于 N 本身的继承属性。
- 终结符号只有综合属性（由词法分析提供），没有继承属性。

SDD示例：表达式求值 (P.15--P.19)

编号	产生式	语义规则
1	$L \rightarrow E \mathbf{n}$	$L.val = E.val$
2	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3	$E \rightarrow T$	$E.val = T.val$
4	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5	$T \rightarrow F$	$T.val = F.val$
6	$F \rightarrow (E)$	$F.val = E.val$
7	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

所有属性均为综合属性，适合自底向上分析。输入 $3 * 5 + 4\mathbf{n}$ 时， $L.val = 19$ (P.19)。

注释语法分析树 (P.16--P.18)

- 在分析树每个结点上标注对应属性值的树，称为注释语法分析树（Annotated Parse Tree）。
- 构造：建分析树 $\rightarrow$ 按产生式应用语义规则 $\rightarrow$ 若 $N.a = f(N_1.b_1, \dots)$ ，需先算出各依赖属性。
- S属性SDD可按自底向上方式求值；存在循环依赖则无法计算。

消除左递归后的SDD (P.20--P.23)

消除左递归后引入继承属性（以乘法文法为例）：

产生式	语义规则
$T \rightarrow F T'$	$T'.inh = F.val; T.val = T'.syn$
$T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val; T'.syn = T'_1.syn$
$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

注： $T'.inh$ 继承了对应 $*$ 号左操作数的值 (P.22)。

5.2 SDD的求值顺序

5.2.1 依赖图 (Dependency Graph) (P.25--P.27)

- 描述某棵分析树上各属性实例之间的计算顺序。
- 有向边 $a_1 \rightarrow a_2$ ：计算 a_2 时需要 a_1 （先算 a_1 ）。
- 构造方法：
 1. 对分析树每个结点 n ，为其每个属性 a 建一个依赖图结点。
 2. 对每条语义规则 $b := f(c_1, \dots, c_k)$ ，从每个 c_i 到 b 连有向边。

属性值的计算顺序 (P.28)

- 按依赖图的拓扑排序计算各属性值。
- 若依赖图含环，则无法计算。
- 特定类型SDD保证无环：S属性SDD 和 L属性SDD。

5.2.3 S属性SDD (P.29--P.30)

- 定义：每个属性均为综合属性。
- 依赖图总是从子结点流向父结点，可在自底向上或自顶向下分析时同步计算。
- 计算方式：后序遍历 (postorder) ——先递归处理所有子结点，再对当前结点各属性求值。
- 在LR分析中，归约时执行语义动作，无需实际构造分析树结点。

5.2.4 L属性SDD (P.31--P.32)

- 定义：每个属性要么是综合属性，要么是继承属性且满足：对产生式 $A \rightarrow X_1 X_2 \cdots X_n$ ，计算 X_i 的继承属性的规则只能使用：
 1. A 的继承属性；
 2. X_j ($j < i$ ，即 X_i 左边) 的继承/综合属性；
 3. X_i 自身的继承/综合属性 (不形成环)。
- 特点：依赖图的边总是从左到右、从下到上。
- 注：S属性SDD $\subseteq$ L属性SDD。

典型例子——变量声明的继承属性 (P.32, P.34)：

产生式	语义规则
$D \rightarrow T L$	$L.inh = T.type$
$T \rightarrow \text{int}$	$T.type = \text{integer}$
$T \rightarrow \text{float}$	$T.type = \text{float}$
$L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh; \text{addType}(\text{id.entry}, L.inh)$
$L \rightarrow \text{id}$	$\text{addType}(\text{id.entry}, L.inh)$

5.2.5 受控副作用 (P.33--P.34)

- 受控副作用：不对属性求值产生约束 (可按任意拓扑顺序求值，结果相同)。
- 例： $L \rightarrow E \text{ n: } \text{print}(E.val)$ ——副作用打印值，不影响其他属性。
- addType ：把标识符类型信息加入符号表，只要不重复声明结果正确。

5.3 语法制导的翻译方案 (SDT)

SDT定义 (P.35)

SDT (Syntax-Directed Translation Scheme) = 在产生式体中嵌入语义动作的上下文无关文法。

- 基本实现：建立语法分析树，从左到右、深度优先地执行语义动作。
- 两类重要SDT：
 - 基本文法是LR的 + SDD是S属性的 $\Rightarrow$ 后缀SDT
 - 基本文法是LL的 + SDD是L属性的 $\Rightarrow$ L属性SDT

产生式内部语义动作 (P.42)

- $B \rightarrow X\{a\}Y$: 动作 a 在 X 处理完成后立即执行。
- 自底向上: a 在 X 出现在栈顶时执行。
- 自顶向下: 在试图展开 Y 或检测到 Y 时执行 a 。

5.3.1 后缀翻译方案 (P.37--P.41)

- 条件: 文法可自底向上分析 + SDD是S属性的。
- 所有语义动作放在产生式最右端, 归约时执行。

产生式	语义动作
$L \rightarrow E \mathbf{n}$	$\{ \text{print}(E.val); \}$
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val; \}$
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$
$F \rightarrow \mathbf{digit}$	$\{ F.val = \mathbf{digit.lexval}; \}$

语法分析栈实现 (P.39--P.41) :

- LR分析栈中每个符号/状态附带属性值 (union结构)。
- 按 $A \rightarrow XYZ$ 归约时: Z 在 $\text{stack}[\text{top}]$, Y 在 $\text{stack}[\text{top}-1]$, X 在 $\text{stack}[\text{top}-2]$ 。
- 例: $E \rightarrow E_1 + T$: $\text{stack}[\text{top}-2].val = \text{stack}[\text{top}-2].val + \text{stack}[\text{top}].val$; $\text{top} -= 2$;

L属性SDD转SDT (P.43--P.44)

- 计算非终结符号 A 继承属性的动作: 放在产生式中紧靠 A 之前。
- 计算产生式头的综合属性的动作: 放在产生式的最右端。

示例 (while语句, P.44) :

$S \rightarrow \mathbf{while} (\underbrace{\{ L_1 = \text{new}(); L_2 = \text{new}(); C.\text{false} = S.\text{next}; C.\text{true} = L_2; \}}_{\text{紧靠C之前}} C) \underbrace{\{ S_1.\text{next} = L_1; \}}_{\text{紧靠}S_1\text{之前}} S_1 \underbrace{\{ S.\text{code} = \dots; \}}_{\text{最右端}} ;$

L属性SDD的递归下降实现 (P.45)

- 每个非终结符号对应一个函数。
- 函数参数接受继承属性, 返回值包含综合属性。
- 函数体: 选产生式 $\rightarrow$ 局部变量保存属性 $\rightarrow$ 终结符读词法属性 $\rightarrow$ 非终结符递归调用并记录返回值。

L属性的自底向上实现 (P.46--P.50)

- 以LL文法为基础的L属性SDD可以在LR语法分析中实现。
- 方法:
 1. 构造L属性SDD的SDT (在非终结符前计算继承属性)。
 2. 对产生式 $A \rightarrow \alpha\{a\}\beta$ 中每个语义动作 a , 引入标记非终结符号 M , 令 $M \rightarrow \varepsilon$ 。
 3. 定义 $M \rightarrow \varepsilon\{a'\}$: 把 a 中需要的 A 等属性拷贝为 M 的继承属性; 按 a 中规则计算结果作为 M 的综合属性。
- 关键: A 的继承属性在执行 M 的归约时, 位于 M 下方的栈记录中。

例5.26 (while语句, P.48--P.50) : 转换后 $S \rightarrow \mathbf{while} (M C) N S_1$

- 归约到 M 时: $S.\text{next}$ 在 $\text{stack}[\text{top}-3].\text{next}$; $C.\text{true}$ 、 $C.\text{false}$ 存放在 M 的栈记录中。
- $S_1.\text{next}$ 存放在 N 的栈记录中 (恰位于 S_1 栈记录之下) : $S_1.\text{next} = \text{stack}[\text{top}-3].L1$ 。

核心概念速查表

概念	关键特征	适合的分析方式
S属性SDD	仅综合属性	自底向上 (LR) , 后序遍历
L属性SDD	综合属性 + 顺序约束继承属性	LL递归下降 / LR+标记符
后缀SDT	动作在产生式最右端	LR分析, 归约时执行
依赖图	有向图, 边表示计算依赖	拓扑排序求值; 有环不可算
标记非终结符	$M \rightarrow \varepsilon$, 物化继承属性计算	L属性自底向上实现

易考点提示

1. 判断综合/继承属性: 规则右侧属于子结点 $\Rightarrow$ 综合; 属于父/兄弟 $\Rightarrow$ 继承。
2. 判断S/L属性SDD: S属性: 全为综合属性。L属性: 继承属性只依赖左边兄弟和父亲。
3. 构造依赖图: 为每个属性实例建结点, 规则 $b = f(c_1, \dots)$ 时从 c_i 到 b 连边; 拓扑排序 = 合法计算顺序。
4. 消除左递归后用继承属性: 对 $T \rightarrow FT'$, 用 $T'.inh$ 把左侧已算出的值传给右边。
5. 后缀SDT的栈实现: 归约产生式 $A \rightarrow X_1 \cdots X_n$, X_i 在 `stack[top-n+i]`, 归约后结果放在 `stack[top-n+1]`。
6. 标记非终结符: L属性自底向上实现的核心, 把动作"物化"为 $M \rightarrow \varepsilon$ 的归约, 提前在归约时取到继